Compte rendu détaillé — ByzFL : prise en main et expérience MNIST (IID)

Introduction

Ce document décrit précisément ce que fait ByzFL, les agrégateurs et attaques que nous avons rencontrés, puis détaille l'expérience MNIST que nous venons d'exécuter (code, protocole, paramètres, résultats et interprétation).

1 Qu'est-ce que ByzFL ?

ByzFL est une bibliothèque Python pour simuler l'apprentissage fédéré robuste face à des clients malveillants (adversaires byzantins). Elle fournit des composants pour :

— définir et répartir des jeux de données entre clients (`DataDistributor`),
— instancier des clients honnêtes qui entraînent localement des modèles (`Client`),
— instancier des clients malveillants qui génèrent des mises à jour corrompues (`ByzantineClient`),
— centraliser l'agrégation et la mise à jour du modèle global (`Server`),
— appliquer différents agrégateurs robustes et pré-agrégateurs avant l'agrégation.

ByzFL supporte les tenseurs PyTorch et les tableaux NumPy et propose de nombreux algorithmes d'attaque et d'agrégation implémentés et testés.

2 Agrégateurs incontournables

— **Average (moyenne simple)** : moyenne coordonnée par coordonnée des vecteurs ; baseline non-robuste.
— **Median (médiane)** : prend la médiane par coordonnée ; robuste aux outliers sur certaines coordonnées.
— **TrMean (Trimmed Mean)** : retire une fraction des valeurs extrêmes avant de faire la moyenne ; paramétré par `f` (nombre d'attaquants) généralement.
— **Krum** : pour chaque vecteur, calcule la somme des distances aux autres vecteurs et sélectionne le vecteur ayant la plus petite somme (robuste dans certains cadres adverses).
— **MultiKrum** : variante de Krum qui retourne plusieurs vecteurs puis les combine.
— **GeometricMedian** : calcule la médiane géométrique (optimise la somme des distances) ; robuste globalement.
— **CenteredClipping** : effectue un clipping centralisé pour limiter l'impact des valeurs extrêmes.
— **MDA / MoNNA / Meamed / CAF / SMEA** : autres méthodes robustes proposées par ByzFL (approches basées sur filtrage statistique, optimisation convexe, ou heuristiques avancées).

Pré-agrégateurs utiles

— **NNM (Nearest Neighbor Mixing)** : mélange voisinage pour atténuer effets extrêmes.
— **Bucketing** : regroupe coordonnées par buckets pour réduire variance.
— **Static Clipping / ARC** : clipping statique ou adaptatif pour limiter amplitude des vecteurs.

3 Attaques byzantines

— **SignFlipping** : renvoie l'opposé de la moyenne des vecteurs honnêtes, simple mais efficace contre les moyennes.
— **InnerProductManipulation (IPM)** : multiplie la moyenne par -τ (ou autre facteur) pour maximiser l'effet sur le produit scalaire, ciblant la direction d'optimisation.
— **ALittleIsEnough (ALIE)** : ajoute τ fois l'écart-type coordinate-wise à la moyenne, cherchant à rester dans la distribution mais à perturber suffisamment.
— **Gaussian** : génère un vecteur aléatoire (iid) selon une loi normale, attaque stochastique.
— **Optimal variants (Opt-IPM, Opt-ALIE)** : versions optimisées des attaques ci-dessus suivant critères de perturbation.
— **Mimic, Infinity, etc.** : autres stratégies décrites dans la doc (mimic tente d'imiter la distribution honnête, Infinity met des entrées très grandes, etc.).

4 Description détaillée de l'expérience réalisée

Objectif : valider localement ByzFL sur MNIST et produire un premier benchmark reproductible comparant quelques agrégateurs face à des attaques.

Fichiers impliqués

— Script : `code/simulation.py`
— Résultats CSV : `code/benchmark_results.csv`
— Compte rendu : `code/compte_rendu_experience_byzfl.txt` (ce fichier)

Paramètres choisis

— Seed : 42 (reproductibilité)
— Nombre de clients honnêtes : 3
— Nombre de clients byzantins : 1
— Batch size : 64
— Nombre de rounds (itérations globales) : 3
— Device : CPU
— Distribution des données : IID (paramètre `data_distribution_name = "iid"`) — tu as modifié pour respecter la contrainte demandée

Pipeline précis (ce que fait le script, fonction par fonction)

1. `resolve_data_root()` : recherche l'emplacement du dossier `data` et vérifie que MNIST est présent.
2. `build_dataloaders()` : charge MNIST via `torchvision.datasets`, applique la normalisation, crée les `DataLoader` train/test et appelle `DataDistributor` pour obtenir un `DataLoader` par client selon la distribution IID.
3. `aggregator_parameters(name)` : prépare les paramètres nécessaires pour chaque agrégateur (ex. `f` pour TrMean/Krum/MultiKrum).
4. `attack_parameters(name)` : prépare les paramètres d'attaque (ex. `tau` pour IPM/A-LIE).
5. `build_server(test_loader, aggregator_name)` : instancie un `Server` ByzFL configuré avec le modèle `cnn_mnist`, l'optimiseur, le test loader et l'agrégateur.
6. `build_clients(client_loaders)` : instancie un `Client` ByzFL pour chaque partition locale.
7. `run_scenario(name, aggregator_name, attack_name)` : boucle principale — synchronise l'état du serveur vers les clients, demande aux clients honnêtes de calculer leurs gradients, génère les vecteurs byzantins via `ByzantineClient.apply_attack()` si nécessaire, combine les vecteurs et appelle `server.update_model_with_gradients()`.
8. `save_results(rows)` : écrit un CSV avec les mesures initiales et finales pour chaque scenario.

5 Scénarios exécutés

Les scenarios comparés sont :
— `clean_average` : baseline propre (aucune attaque) avec `Average` ;
— `signflip_average` : `SignFlipping` vs `Average` ;
— `signflip_median` : `SignFlipping` vs `Median` ;
— `ipm_trmean` : `InnerProductManipulation` vs `TrMean` ;
— `alie_krum` : `ALittleIsEnough` vs `Krum`.

6 Résultats (exécution finale, distribution IID)

Le CSV `code/benchmark_results.csv` contient les valeurs suivantes (vues lors du dernier run) :

scenario	aggregator	attack	initial_accuracy	final_accuracy
clean_average	Average	clean	0.0600	0.1217
signflip_average	Average	SignFlipping	0.1042	0.1202
signflip_median	Median	SignFlipping	0.1068	0.1161
ipm_trmean	TrMean	InnerProductManipulation	0.1173	0.1305
alie_krum	Krum	ALittleIsEnough	0.0976	0.1041

Notes sur les mesures

— `initial_accuracy` : accuracy calculée avant les rounds (évalue l'état initial du serveur) ;
— `final_accuracy` : accuracy après les `NB_ROUNDS` (ici 3) rounds.

7 Analyse et interprétation

— Ces résultats sont obtenus sur une expérience courte (3 rounds) et sur CPU : les valeurs numériques sont modestes et servent principalement à valider la chaîne d'exécution.
— On observe que `TrMean` (avec IPM) atteint la meilleure `final_accuracy` (0.1305) dans ce petit test, mais ce n'est pas suffisant pour conclure sur la robustesse réelle : il faut évaluer sur plus d'itérations, plus d'essais (moyenne), et varier `f`.
— `Krum` avec ALIE montre ici une amélioration faible (0.1041), ce qui peut indiquer que le paramétrage (`f`, nombre de clients) influence fortement l'efficacité.
— `Median` et `TrMean` sont des techniques classiques résistant aux outliers ; IPM et ALIE ciblent différemment ces mécanismes, d'où l'importance de tester plusieurs attaques.

8 Limitations de cette expérience

— Trop peu de rounds : 3 rounds ne suffisent pas pour observer une convergence réelle.
— Petit nombre de clients : 3 clients limitent la diversité et l'effet de certaines défenses comme Krum.
— Mesures non moyennées : il faut lancer plusieurs seeds/exécutions pour obtenir des résultats statistiquement signifiants.

9 Références

— Documentation ByzFL : `https://byzfl.epfl.ch/`